

Nomes: João Vitor Curcio Sutter e Pedro Henrique Gimenez
Disciplina: INE5202 - Cálculo Numérico em Computadores
Linguagem escolhida: Python 3.10.12

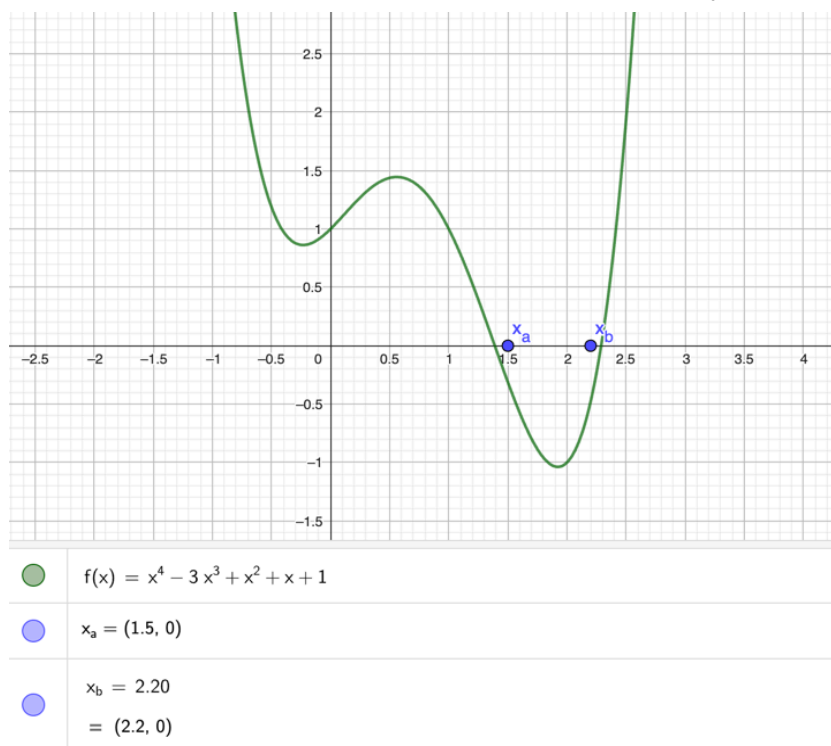
Relatório - Exercício Programa I

1 - Método de Horner

1.1 - Introdução

Na tarefa 1, deve-se implementar o método de Horner para aproximação de raízes. A função utilizada para testar o algoritmo é o polinômio $f(x) = x^4 - 3x^3 + x^2 + x + 1$, cujo gráfico pode ser visualizado na figura 1:

Figura 1 - Gráfico do polinômio com os pontos x_0



Fonte: GeoGebra Graphing Calculator

Conforme computado pelo GeoGebra, as raízes se encontram próximas aos pontos $x_a(1.5, 0)$ e $x_b(2.2, 0)$. Sendo assim, para os valores de x_0 , foram escolhidos 1.5 e 2.2, respectivamente.

1.2 - Codificação do algoritmo em Python

Para implementar o algoritmo de Horner na linguagem Python, criamos uma função chamada *horner* que recebe o grau (int), os coeficientes (lista de inteiros) e um x_0 (int ou float).

Começamos definindo y e z como a_n e depois fomos iterando num *for-loop* que vai de $n - 1$ até 1. Em cada iteração, temos $y = x_0 \cdot y + a_n$ e $z = x_0 \cdot z + y$.

Para facilitar a visualização do algoritmo, imprimimos no console o valor de y e z em cada iteração. Por fim, para finalizar o algoritmo, definimos $y = x_0 \cdot y + a_0$ e imprimimos os valores finais de y e z , bem como o valor obtido para a raiz encontrada: $x_1 = x_0 - \frac{y}{z}$.

1.3 - Aplicação do algoritmo

Para achar a primeira raiz do polinômio $f(x) = x^4 - 3x^3 + x^2 + x + 1$, chamamos a função *horner* com os parâmetros grau = 4, coeficientes = [1, 1, 1, -3, 1] e $x_0 = 1.5$ (escolhido com base no gráfico mostrado na introdução)

Ao rodar a função, obtivemos:

Figura 2 - Console após compilar a função *horner* em $f(x)$ com $x_0 = 1.5$

```
Iniciando algoritmo de Horner...

----- VALORES DE Y E Z -----
-----
y = 1
z = 1
-----
y = -1.5
z = 0.0
-----
y = -1.25
z = -1.25
-----
Valor final de y = -0.3125
Valor final de z = -2.75
Valor obtido para x1: 1.3863636363636365
```

Fonte: De autoria própria

A função indicou que o valor da primeira raiz x_1 é aproximadamente 1.38636.

Em seguida, chamamos a função *horner* novamente para obter a segunda raiz x_2 e obtivemos:

Figura 3 - Console após compilar a função *horner* em $f(x)$ com $x_0 = 2.2$

```
Iniciando algoritmo de Horner...

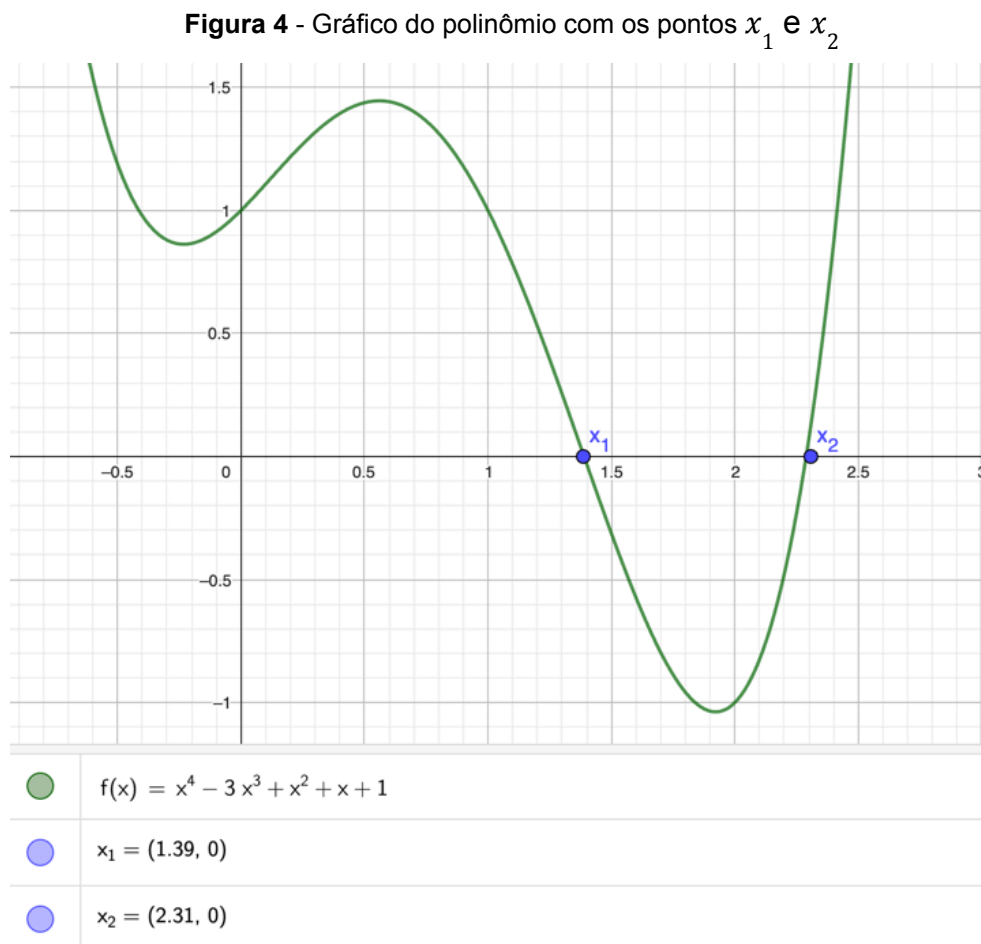
----- VALORES DE Y E Z -----
-----
y = 1
z = 1
-----
y = -0.7999999999999998
z = 1.4000000000000004
-----
y = -0.7599999999999998
z = 2.3200000000000001
-----
Valor final de y = -0.4783999999999995
Valor final de z = 4.4320000000000003
Valor obtido para x1: 2.307942238267148
```

Fonte: De autoria própria

O programa indicou que a segunda raiz x_2 é aproximadamente 2.30794.

1.4 - Análise dos resultados

Com os resultados fornecidos pelas funções, inserimos no gráfico do polinômio $f(x)$ os pontos $x_1(1.38636, 0)$ e $x_2(2.30794, 0)$, como pode ser observado na figura 4.



Fonte: GeoGebra Graphing Calculator

Podemos inferir, após observar o gráfico, que o algoritmo foi executado com sucesso, retornando pontos bem próximos da raiz. É interessante ressaltar também que se aplicarmos o algoritmo novamente com os pontos obtidos, a raiz se torna mais precisa ainda.

2 - Método de Muller

2.1 - Introdução

Na tarefa 2, deve-se implementar o método de Muller para aproximação de raízes. A função utilizada para testar o algoritmo é o mesmo polinômio da tarefa 1

$$f(x) = x^4 - 3x^3 + x^2 + x + 1.$$

Para aplicar o método de Muller, escolhemos as aproximações (1.2, 1.3, 1.5) para a primeira raiz real, (2.2, 2.1, 2.4) para a segunda raiz real e as aproximações fornecidas pela tarefa (-0.5, 0, 0.5) para a raiz complexa.

2.2 - Codificação do método em Python

Para implementar o método de Muller na linguagem Python, criamos uma função chamada *muller* que possui como parâmetros o polinômio e as aproximações x_0 , x_1 e x_2 .

Dentro da função, definimos a tolerância (valor que define se a aproximação está próxima da raiz o suficiente) para 0.01 e o número máximo de iterações para 1000.

Em seguida, traduzimos o algoritmo em pseudocódigo fornecido pelo exercício para linguagem Python.

2.3 - Aplicação do método

Para achar as raízes reais, chamamos a função *muller* fornecendo os parâmetros: polinômio $f(x)$ e as aproximações (1.2, 1.3, 1.5) e (2.2, 2.1, 2.4) escolhidas no item 2.1.

Após o programa ser executado, ele nos fornece as seguintes raízes no console:

Figura 5 - Console após compilar a função *muller* em $f(x)$ com aproximações das raízes reais

```
Iniciando algoritmo de Muller...  
O algoritmo foi executado com sucesso!  
Valor aproximado da raiz: 1.390970657505983  
  
Iniciando algoritmo de Muller...  
O algoritmo foi executado com sucesso!  
Valor aproximado da raiz: 2.2887611530598915
```

Fonte: De autoria própria

Em seguida, chamamos a função *muller* novamente para achar a raiz complexa a partir das aproximações (-0.5, 0, 0.5) fornecidas pelo exercício. O resultado pode ser visto na figura 6.

Figura 6 - Console após compilar a função *muller* em $f(x)$ com aproximações da raiz complexa

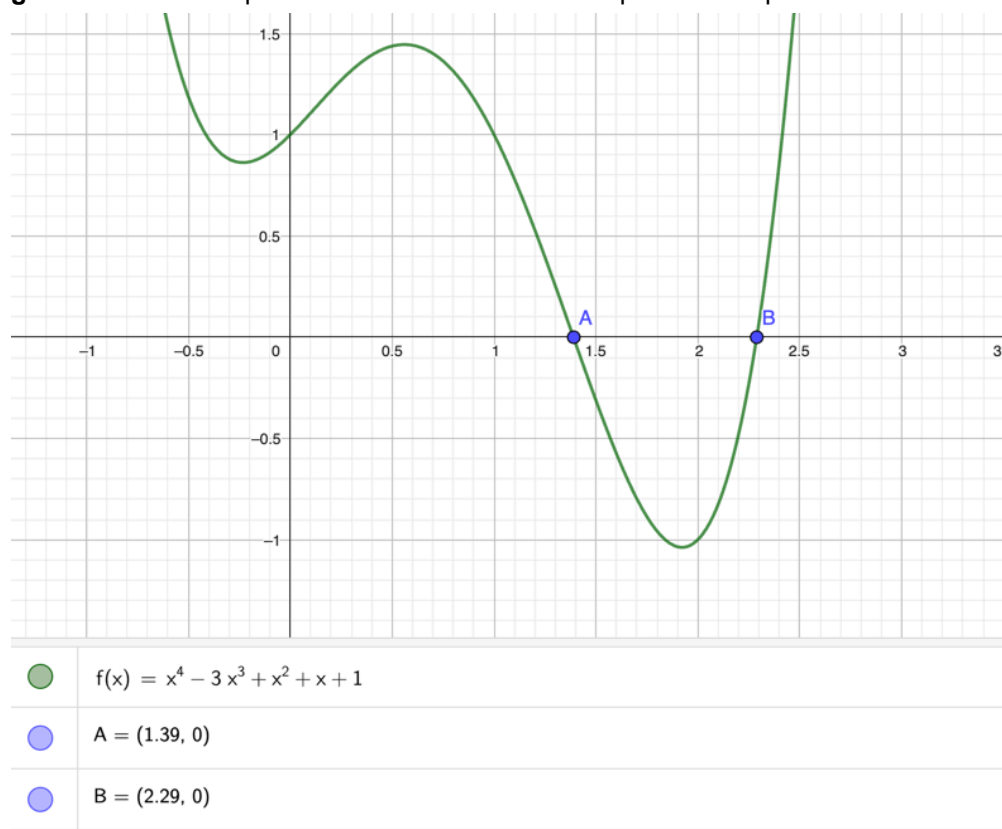
```
Iniciando algoritmo de Muller...  
O algoritmo foi executado com sucesso!  
Valor aproximado da raiz: (-0.3398219720143227+0.4446669244054705j)
```

Fonte: De autoria própria

1.4 - Análise dos resultados

Com os resultados obtidos pela função, inserimos os pontos aproximados das raízes reais no gráfico do polinômio utilizando a ferramenta Geogebra, como pode ser observado na figura 7.

Figura 7 - Gráfico do polinômio com as raízes reais aproximadas pelo método de Muller



Fonte: GeoGebra Graphing Calculator

Analisando o gráfico, podemos perceber que os pontos obtidos utilizando o método de Muller estão bem próximos dos valores das raízes reais, indicando que o algoritmo foi realizado com sucesso.